

BORDERLESS TECH ACADEMY

CI/CD Pipelines & Automation Mastery

A Comprehensive Training Guide

From Core Concepts to GitHub Actions Mastery

Jenkins · GitLab CI/CD · AWS CodePipeline · GitHub Actions

Version 1.0 | April 2026

PART 1: UNDERSTANDING CI/CD

Chapter 1: What Is CI/CD and Why Does It Matter?

1.1 The Problem CI/CD Solves

Before CI/CD existed, software teams faced what developers called the 'integration hell.' Imagine ten developers working independently for two weeks, each changing hundreds of lines of code. When it was finally time to merge everyone's work together, chaos ensued — conflicts, broken features, and bugs that were nearly impossible to trace. Releases took days of manual testing and deployment, and a single mistake could take down the production server.

The Old Way (Without CI/CD)

- Developers work in isolation for weeks or months
- Code is merged all at once — 'big bang' integration
- Bugs discovered late are expensive and difficult to fix
- Manual testing is slow, inconsistent, and error-prone
- Deployments are high-risk, done infrequently, often on Friday evenings
- The feedback loop from code to production can take weeks

1.2 What CI/CD Actually Means

Continuous Integration (CI)

Continuous Integration is the practice of automatically integrating code changes from multiple developers into a shared repository frequently — ideally multiple times per day. Every integration is verified by an automated build and automated tests.

- Developers commit small, focused changes often
- Every commit triggers an automated pipeline
- Tests run automatically to detect issues immediately
- Broken builds are fixed before moving forward

Continuous Delivery (CD)

Continuous Delivery extends CI by ensuring that code is always in a deployable state. After passing all tests, the software can be deployed to production at any time with the push of a button. The key word is 'can' — a human still decides when to deploy.

Continuous Deployment (Also CD)

Continuous Deployment goes one step further: every change that passes all automated tests is automatically deployed to production with no human intervention required. This is the most advanced maturity level of CI/CD.

Term	Definition
Continuous Integration	Automatically test & build every commit
Continuous Delivery	Software is always deployable; release is manual decision
Continuous Deployment	Every passing commit goes to production automatically

1.3 The Core Benefits of CI/CD

1. Faster Feedback Loops

When a developer pushes code, CI runs in minutes and reports back whether the build passed or failed. Compare this to finding a bug weeks later — the cost to fix it grows exponentially the later it is discovered. Studies show bugs caught in CI cost 6x less to fix than those found in QA, and 30x less than those found in production.

2. Reduced Risk Per Release

Small, frequent releases are dramatically safer than large, infrequent ones. If a bug appears in a deployment that changed 50 lines of code, you know exactly where to look. A deployment that changed 50,000 lines is a nightmare to debug.

3. Improved Team Collaboration

CI/CD forces developers to integrate their work regularly, which surfaces conflicts early and encourages communication. It also creates a shared understanding of the codebase's health through visible build dashboards.

4. Higher Code Quality

Automated testing, code linting, security scans, and code coverage checks all run automatically in a CI pipeline. This creates an objective quality gate that every change must pass, regardless of who wrote it.

5. Faster Time to Market

When deployment is automated and reliable, the business can ship features faster. Competitive advantage in modern tech is often measured in days, not months.

1.4 Implications of Not Having CI/CD

Business & Technical Risks Without CI/CD

Security vulnerabilities stay in production longer due to slow deployment cycles

Talent risk: top engineers avoid teams with poor DevOps culture

Customer trust erodes when bugs reach production frequently

Technical debt accumulates because manual processes slow refactoring

Competitive disadvantage — competitors shipping weekly vs your quarterly releases

High burnout among developers managing manual, repetitive deployment tasks

Chapter 2: The Stages of a CI/CD Pipeline

A CI/CD pipeline is a sequence of automated stages, each performing a specific quality or deployment function. Think of it as an assembly line for software — code enters one end as a raw commit and exits the other end as a deployed, validated release.

Stage 1: Source Control Trigger

The pipeline begins when code is pushed to a repository (GitHub, GitLab, Bitbucket, AWS CodeCommit). This trigger is the entry point for all automation that follows. Common triggers include:

- Push to a feature branch
- Pull Request / Merge Request opened or updated
- Merge to the main/master branch
- A scheduled cron trigger (e.g., nightly builds)
- A manual workflow dispatch

Stage 2: Build

The build stage compiles or packages the source code into a runnable artifact. What this looks like depends on the technology stack:

- Node.js: `npm install && npm run build`
- Java/Maven: `mvn clean package`
- Python: `pip install && pyinstaller` or `Docker build`
- Docker: `docker build -t myapp:latest .`

If the build fails, the pipeline stops immediately. No point testing code that doesn't compile.

Stage 3: Test

This is the most critical stage for quality assurance. Testing typically runs in multiple layers:

Unit Tests

Test individual functions or components in isolation. Fast (milliseconds each), and developers should have hundreds or thousands of these. Tools: Jest, Pytest, JUnit, RSpec.

Integration Tests

Test how components work together — API endpoints hitting a database, services communicating over HTTP. Slower than unit tests. Tools: Supertest, Postman/Newman, REST Assured.

End-to-End (E2E) Tests

Simulate real user interactions through the UI. Slowest but most realistic. Tools: Cypress, Playwright, Selenium.

Code Coverage

Measure what percentage of your code is exercised by tests. A coverage threshold (e.g., 80%) can be set as a required gate.

Stage 4: Static Analysis & Code Quality

Before code gets deployed, automated tools analyze it for issues that aren't runtime errors:

- Linting: Enforce code style rules (ESLint, Flake8, RuboCop)
- Static analysis: Detect code smells and complexity issues (SonarQube, CodeClimate)
- Dependency auditing: Check for vulnerable packages (npm audit, OWASP Dependency-Check)
- Type checking: TypeScript type validation, mypy for Python

Stage 5: Security Scanning (SAST/DAST)

Security must be shifted left — caught early in the pipeline, not after deployment.

- SAST (Static Application Security Testing): Analyze source code for security vulnerabilities without running it. Tools: Snyk, Semgrep, CodeQL
- DAST (Dynamic Application Security Testing): Test a running application for vulnerabilities like SQL injection, XSS. Tools: OWASP ZAP, Burp Suite
- Secret scanning: Ensure no API keys, passwords, or credentials were accidentally committed

Stage 6: Artifact Creation

Once code passes all tests, it's packaged into a deployable artifact:

- Docker image: Built and pushed to a registry (Docker Hub, Amazon ECR, GitHub Container Registry)
- Binary/executable: Stored in an artifact repository (Nexus, JFrog Artifactory, GitHub Releases)
- Infrastructure package: Terraform plan files, CloudFormation templates

Stage 7: Deployment to Staging

The artifact is deployed to a staging environment — a production-like environment used for final validation. Smoke tests, performance tests, and user acceptance testing (UAT) happen here. This is the last line of defense before production.

Stage 8: Deployment to Production

The final stage deploys to the live environment. Advanced deployment strategies minimize risk:

- Blue/Green Deployment: Run two identical environments; switch traffic from old (blue) to new (green)
- Canary Deployment: Route a small percentage (1-5%) of traffic to the new version first; roll out gradually
- Rolling Deployment: Replace instances one by one, maintaining availability throughout
- Feature Flags: Deploy code but control who sees new features via configuration

Stage 9: Monitoring & Observability

After deployment, the pipeline doesn't truly end. Monitoring ensures the deployment was successful and production remains healthy:

- Metrics: CPU, memory, request latency, error rates (Datadog, Prometheus, CloudWatch)
- Logging: Centralized log aggregation (ELK Stack, CloudWatch Logs, Datadog Logs)
- Alerting: Automated alerts when SLOs are breached
- Automated rollback: If error rate spikes post-deployment, automatically revert

PART 2: CI/CD TOOLS OVERVIEW

Chapter 3: Common CI/CD Tools Compared

The CI/CD tooling landscape is rich. Each tool has trade-offs in complexity, cost, flexibility, and integration. Understanding these differences will help you choose the right tool for the right context.

3.1 Jenkins

What Is Jenkins?

Jenkins is the grandfather of CI/CD — an open-source automation server first released in 2011. It's self-hosted, meaning you install and maintain it on your own servers. Jenkins has a massive ecosystem of over 1,800 plugins.

Strengths

- Free and open source
- Extremely flexible — can be configured to do almost anything
- Huge plugin ecosystem (Docker, Kubernetes, Slack, SonarQube, etc.)
- Supports both scripted and declarative pipeline syntax (Jenkinsfile)
- Battle-tested in enterprise environments

Weaknesses

- High operational overhead — you manage servers, updates, security patches
- Configuration UI is dated; pipeline-as-code requires learning Groovy
- Plugin conflicts are a common source of pain
- Scaling requires expertise (Jenkins master/agent architecture)
- No built-in secret management

```
// Jenkinsfile (Declarative Pipeline) - Example
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'npm install'
                sh 'npm run build'
            }
        }
    }
}
```



```
    stage('Test') {
      steps {
        sh 'npm test'
      }
    }
    stage('Deploy') {
      steps {
        sh './deploy.sh'
      }
    }
  }
  post {
    failure {
      mail to: 'team@company.com',
          subject: 'Build Failed: ${currentBuild.fullDisplayName}'
    }
  }
}
```

✓ Best For:

- Enterprise organizations with dedicated DevOps teams
- Complex, highly customized pipelines with unusual requirements
- Organizations with strict compliance needs requiring on-premises infrastructure

3.2 GitLab CI/CD

What Is GitLab CI/CD?

GitLab CI/CD is built directly into the GitLab platform (both SaaS at gitlab.com and self-hosted). Configuration is done via a `.gitlab-ci.yml` file in the repository root. GitLab CI/CD is deeply integrated with GitLab's SCM, issue tracking, and container registry.

Strengths

- Fully integrated with source code, issue tracker, container registry, and package registry
- Built-in Docker registry and Kubernetes integration
- Auto DevOps: automatically detects and configures pipelines for common languages
- Excellent for monorepos and multi-project pipelines
- Built-in security scanning (SAST, DAST, dependency scanning)
- Free tier includes 400 CI/CD minutes per month

Weaknesses

- Tied to GitLab — migration to another platform is costly
- Self-managed option adds operational overhead

- Can be expensive for large teams on paid tiers

```
# .gitlab-ci.yml - Example
stages:
  - build
  - test
  - deploy

build:
  stage: build
  image: node:18
  script:
    - npm install
    - npm run build
  artifacts:
    paths: [dist/]

test:
  stage: test
  image: node:18
  script:
    - npm test

deploy-production:
  stage: deploy
  script:
    - ./deploy.sh
  only:
    - main
```

✓ Best For:

- Teams already using GitLab for source control
- Organizations wanting an all-in-one DevOps platform
- Teams needing built-in security scanning without extra tools

3.3 AWS CodePipeline

What Is AWS CodePipeline?

AWS CodePipeline is a fully managed continuous delivery service from Amazon Web Services. It's one piece of the AWS Developer Tools suite, which includes CodeCommit (SCM), CodeBuild (build service), CodeDeploy (deployment), and CodePipeline (orchestration).

Strengths

- Deep, native integration with all AWS services (ECS, Lambda, S3, Elastic Beanstalk, EC2)
- Fully managed — no servers to maintain

- Visual pipeline editor in AWS Console
- Pay-per-pipeline model (one active pipeline = \$1/month)
- Native IAM integration for fine-grained security control
- Supports multi-region and multi-account deployments natively

Weaknesses

- Vendor lock-in — deeply tied to AWS ecosystem
- Limited flexibility compared to Jenkins or GitHub Actions for non-AWS workloads
- UI-heavy configuration; pipeline-as-code requires CloudFormation/CDK/Terraform
- CodeBuild can be slower to start (container spin-up time) vs GitHub-hosted runners

✓ Best For:

Organizations deeply invested in the AWS ecosystem (Imisi Health, for example)

Teams deploying to ECS, Lambda, EC2, or Elastic Beanstalk

Enterprises requiring compliance with AWS-native security controls and auditing

3.4 GitHub Actions

What Is GitHub Actions?

GitHub Actions is GitHub's built-in CI/CD and automation platform, launched in 2019. It allows you to automate workflows directly in your GitHub repository using YAML files stored in `.github/workflows/`. It has rapidly become the most popular CI/CD tool for open-source and startup teams.

Strengths

- Zero infrastructure to manage (GitHub-hosted runners)
- Deeply integrated with GitHub pull requests, issues, releases, and packages
- Massive marketplace of reusable Actions (15,000+ community actions)
- Matrix builds: easily test across multiple OS/language version combinations
- Free for public repos; generous free tier for private repos
- Supports custom self-hosted runners for special requirements
- OIDC support for keyless authentication to cloud providers

Weaknesses

- Tightly coupled to GitHub — not ideal if you use Bitbucket or GitLab
- Complex workflows with many jobs can become difficult to read

- Self-hosted runners require maintenance if used
- Secrets management is basic compared to HashiCorp Vault or AWS Secrets Manager

3.5 Side-by-Side Comparison

Feature	Jenkins	GitLab CI/CD	GitHub Actions	AWS CodePipeline
Infrastructure	Self-hosted	Self-hosted/SaaS	GitHub-hosted/Self	Fully managed AWS
Config format	Groovy (Jenkinsfile)	YAML (.gitlab-ci.yml)	YAML (.github/workflows/)	Visual + JSON/CDK
Learning curve	High	Medium	Low-Medium	Medium-High
Cloud provider	Cloud-agnostic	Cloud-agnostic	Cloud-agnostic	AWS only
Cost model	Free (infra cost)	Per-minute / user	Per-minute / repo	Per pipeline/min
Community/Marketplace	1,800+ plugins	400+ templates	15,000+ actions	AWS integrations
Secret management	Plugin-based	Built-in	Repository secrets	AWS Secrets Manager
Best for	Enterprise/complex	GitLab users	GitHub users/startups	AWS-centric teams

PART 3: GITHUB ACTIONS — COMPREHENSIVE TRAINING

Chapter 4: GitHub Actions Fundamentals

GitHub Actions is the CI/CD tool you'll use most as a modern developer working on GitHub-hosted projects. This section is a complete training journey from your first workflow to advanced production patterns.

4.1 Core Concepts & Terminology

Workflow

A workflow is an automated process defined in a YAML file. Stored in `.github/workflows/` in your repository. A repo can have multiple workflows (e.g., one for CI, one for deployment, one for dependency updates).

Event

An event is what triggers a workflow. Examples: `push`, `pull_request`, `schedule`, `workflow_dispatch` (manual), `release`, `issues`. You can combine multiple triggers.

Job

A workflow contains one or more jobs. Each job runs on a fresh virtual machine (runner). By default, jobs run in parallel. You can set dependencies to run them sequentially.

Step

A job contains a sequence of steps. Each step is either a shell command (`run:`) or a reusable Action (`uses:`). Steps run in order within a job and share the same runner environment.

Action

An Action is a reusable unit of code — like a function you call from the GitHub Marketplace or your own repository. Actions can set up Node.js, check out code, deploy to AWS, send Slack notifications, and much more.

Runner

A runner is the server that executes jobs. GitHub provides hosted runners (`ubuntu-latest`, `windows-latest`, `macos-latest`). You can also use self-hosted runners on your own infrastructure.

4.2 Deep Dive: What Are Actions?

The word 'Actions' in GitHub Actions refers to two different things: the platform itself, and the individual reusable components you plug into your workflows. Understanding the component meaning is critical — Actions are what make GitHub's CI/CD system powerful and accessible.

What an Action Actually Is

An Action is a self-contained, reusable piece of automation logic packaged so that any workflow can use it in a single line. Think of it like an npm package, but for CI/CD steps. Instead of writing 30 lines of shell script to set up Node.js, configure caching, and validate the environment, you write one line:

```
- uses: actions/setup-node@v4
  with:
    node-version: '18'
    cache: 'npm'
```

That one action internally sets up the correct Node.js binary, configures PATH, sets up npm/yarn/pnpm caching, and validates everything. The complexity is hidden inside the action — you just consume it.

The Three Types of Actions

Type 1: JavaScript Actions

Written in JavaScript/TypeScript and run directly on the runner using Node.js.

Fastest startup time — no container to spin up.

Can use any npm package. Access to the GitHub Actions toolkit (@actions/core, @actions/github, etc.)

Example: actions/checkout, actions/setup-node, actions/cache

Best for: actions that need to be fast, interact with GitHub's API, or run cross-platform

Type 2: Docker Container Actions

The action runs inside a Docker container, giving complete control over the environment.

Slower startup (container must be pulled/built) but highly reproducible.

Can use any language — Python, Ruby, Go, Rust — anything that runs in Docker.

Only runs on Linux runners (not Windows/macOS).

Best for: actions needing specific system tools, precise OS environments, or non-JS

languages

Type 3: Composite Actions

A sequence of workflow steps bundled into a reusable action — no programming required.

Written in YAML, just like a workflow. Can call other actions and run shell commands.

Easiest to create: if you can write a workflow step, you can write a composite action.

Best for: packaging groups of steps that repeat across multiple workflows in your org

Action Configuration: The 'with' and 'env' Keywords

Actions are configured through inputs passed via the `with:` keyword. Each action documents what inputs it accepts, which are required vs optional, and what their defaults are. You find this documentation either on the GitHub Marketplace page or in the action's repository README.

```
# Full anatomy of calling an action with configuration

steps:
  - name: Setup Node.js           # Human-readable step name
    id: setup                     # Optional: step ID to reference outputs later
    uses: actions/setup-node@v4   # action@version
    with:                          # Inputs to the action
      node-version: '18'         # Input: which Node.js version
      cache: 'npm'               # Input: enable caching for npm
      cache-dependency-path: |   # Input: custom path(s) for cache key
        package-lock.json
        apps/*/package-lock.json
    env:                          # Environment variables scoped to THIS step
      NODE_ENV: production
```

Reading Action Outputs

Many actions produce outputs — values you can read in subsequent steps using the step's id. This is how you chain actions together, passing data from one step to the next.

```
steps:
  # Step 1: Log into ECR, capture the registry URL as an output
  - name: Login to Amazon ECR
    id: login-ecr                 # Assign an ID to reference outputs
    uses: aws-actions/amazon-ecr-login@v2

  # Step 2: Use the output from Step 1
  - name: Build Docker image
    run: |
      REGISTRY=${{ steps.login-ecr.outputs.registry }}
      docker build -t $REGISTRY/my-app:latest .
```

```
# steps.<id>.outputs.<output-name> is the syntax

# Another example: docker/metadata-action produces image tags as an output
- name: Generate image metadata
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ghcr.io/myorg/myapp

- name: Build and push
  uses: docker/build-push-action@v6
  with:
    tags: ${{ steps.meta.outputs.tags }}      # Output from previous step
    labels: ${{ steps.meta.outputs.labels }}
```

How to Version-Pin Actions (and Why It Matters)

When you write `uses: actions/checkout@v4`, the `@v4` is a Git reference — it can be a tag, a branch, or a commit SHA. How you pin your actions has significant security and stability implications.

```
# Option 1: Floating major tag (convenient, but risky)
- uses: actions/checkout@v4
# Tag 'v4' can be moved by the action author at any time
# Automatically gets patch and minor updates – but also breaking changes

# Option 2: Exact version tag (better for stability)
- uses: actions/checkout@v4.1.1
# Pinned to a specific release – won't change without you updating it

# Option 3: Full commit SHA (most secure – recommended for production)
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
# A SHA cannot be moved or tampered with – it's immutable
# Add a comment with the tag name so humans know what version it is
# Tools like Dependabot and Renovate can auto-update these SHAs
```

Security Note on Version Pinning

In 2021, the `'tj-actions/changed-files'` action was compromised — attackers modified the tag

to point to malicious code. Every repo using that action by tag was exposed.

The attack would have been blocked if repos pinned to a commit SHA instead of a tag.

For production workflows, always pin third-party actions to a full commit SHA.

GitHub's own actions (`actions/*`) are safer to use by tag since they're controlled by GitHub.

Where to Find Actions

1. GitHub Marketplace

The primary catalog of published actions. Navigate to github.com/marketplace?type=actions. You can filter by category (CI, deployment, code review, notifications, etc.), sort by star count, and see verified badges on trusted publisher actions. Every action listing shows its inputs, outputs, example usage, and changelog.

2. The 'actions' Organisation on GitHub

GitHub's first-party actions are all under github.com/actions. These are the most trusted and well-maintained actions available. They include the actions you'll use in almost every workflow.

3. Cloud Provider Official Actions

All major cloud providers maintain official action organisations: [aws-actions](https://github.com/aws-actions) (github.com/aws-actions), [google-github-actions](https://github.com/google-github-actions) (github.com/google-github-actions), and Azure's [azure/actions](https://github.com/Azure/actions). These are the canonical way to deploy to each cloud from GitHub Actions.

4. Your Own Repository

You can define actions locally in your own repo under `.github/actions/action-name/action.yml`.

Reference them with a relative path: uses: `./github/actions/my-action`. Ideal for composite actions that encapsulate your team's repeated steps.

5. Any Public GitHub Repository

Any public repo can publish an action. The syntax uses: `owner/repo@ref` works for any repository, not just Marketplace-listed ones. Always review the source code of third-party actions before using them — you're running their code on your infrastructure.

Popular Actions You Will Use Constantly

Action	What It Does
<code>actions/checkout@v4</code>	Checks out your repository code so the workflow can access it. Always your first step.
<code>actions/setup-node@v4</code>	Installs a specific Node.js version and configures npm/yarn/pnpm caching.
<code>actions/setup-python@v5</code>	Installs Python and pip. Supports caching of pip packages.
<code>actions/setup-java@v4</code>	Sets up a JDK version (Temurin, Corretto, etc.) for Java/Kotlin/Scala.
<code>actions/cache@v4</code>	General-purpose caching for any directory or file glob.
<code>actions/upload-artifact@v4</code>	Saves files from a job so other jobs or humans can download them.

actions/download-artifact@v4	Retrieves files saved by upload-artifact in a previous job.
actions/github-script@v7	Run JavaScript inline that calls GitHub's API using the Octokit client.
docker/login-action@v3	Authenticates to Docker Hub, GHCR, ECR, or any registry.
docker/build-push-action@v6	Builds a Docker image and pushes to a registry with full BuildKit support.
docker/metadata-action@v5	Generates Docker image tags and labels from Git metadata.
aws-actions/configure-aws-credentials@v4	Configures AWS credentials using OIDC or access keys.
aws-actions/amazon-ecr-login@v2	Authenticates Docker to Amazon ECR.
aws-actions/amazon-ecs-deploy-task-definition@v2	Deploys a new task definition revision to ECS.
google-github-actions/auth@v2	Authenticates to Google Cloud using Workload Identity Federation (OIDC).
azure/login@v2	Authenticates to Azure using a service principal or OIDC.
codecov/codecov-action@v4	Uploads test coverage reports to Codecov.io.
github/codeql-action/init@v3	Initialises CodeQL for static security analysis.
snky/actions/node@master	Runs Snyk dependency vulnerability scanning for Node.js projects.
slackapi/slack-github-action@v2	Sends messages to a Slack channel via webhook or bot token.
peter-evans/create-pull-request@v6	Automatically creates a pull request from workflow-generated changes.
EndBug/add-and-commit@v9	Stages, commits, and pushes changes made during a workflow.
softprops/action-gh-release@v2	Creates a GitHub Release and uploads release assets.

Writing Your Own Action: Composite Example

Here is a complete example of a custom composite action. This packages a repeated pattern — checking out code, setting up Node, installing deps, and running a build — into a single reusable unit.

```
# File: .github/actions/node-setup-build/action.yml

name: 'Node.js Setup and Build'
description: 'Checks out code, sets up Node.js, installs deps, and builds the project'

inputs:
  node-version:
    description: 'Node.js version to install'
    required: false
    default: '18'
```

```
working-directory:
  description: 'Directory to run commands in'
  required: false
  default: '.'

outputs:
  build-path:
    description: 'Path to the built output'
    value: ${{ steps.set-output.outputs.path }}

runs:
  using: 'composite' # This is a composite action
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: 'npm'

    - name: Install dependencies
      shell: bash # composite actions MUST specify shell
      working-directory: ${{ inputs.working-directory }}
      run: npm ci

    - name: Build project
      shell: bash
      working-directory: ${{ inputs.working-directory }}
      run: npm run build

    - name: Set build path output
      id: set-output
      shell: bash
      run: echo "path=${{ inputs.working-directory }}/dist" >> $GITHUB_OUTPUT

---

# Usage in any workflow file:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: ../github/actions/node-setup-build
        with:
          node-version: '20'
          working-directory: './apps/api'
```

4.3 Deep Dive: Runners

A runner is a server — a virtual machine or physical computer — that receives a job from GitHub, executes every step in that job, and reports the results back. Understanding runners is essential

because the runner type determines cost, performance, network access, and what software is available to your jobs.

GitHub-Hosted Runners (Managed Runners)

GitHub-hosted runners are virtual machines that GitHub provisions, manages, and destroys on your behalf. You declare which runner you want with the `runs-on:` keyword. Every job gets a completely fresh, clean virtual machine — nothing persists between jobs or workflow runs.

Runner Label	Description
ubuntu-latest	Ubuntu 24.04 LTS — fastest, cheapest, most popular. Best default choice.
ubuntu-22.04	Ubuntu 22.04 LTS — pin this if you need a specific Ubuntu version.
ubuntu-20.04	Ubuntu 20.04 LTS — legacy support only; prefer 22.04 or later.
windows-latest	Windows Server 2022 — for .NET, PowerShell, or Windows-specific testing.
windows-2022	Pinned Windows Server 2022 version.
macos-latest	macOS Sonoma — required for iOS/macOS app builds. Most expensive runner.
macos-14	macOS Sonoma on Apple Silicon (M1) — faster than Intel macOS runners.
macos-13	macOS Ventura on Intel — use for Intel-specific macOS builds.

What Comes Pre-Installed on GitHub-Hosted Runners

GitHub-hosted runners come loaded with an extensive software toolkit so you rarely need to install common tools yourself. The `ubuntu-latest` runner includes, among many others:

- Languages: Node.js (multiple versions via `nvm`), Python 3.x, Java (multiple JDKs), Ruby, Go, Rust, PHP, .NET
- Containers: Docker, Docker Compose, Podman
- Cloud CLIs: AWS CLI, Azure CLI, Google Cloud SDK, Heroku CLI
- Databases: PostgreSQL client, MySQL client, SQLite
- Tools: Git, curl, wget, jq, unzip, make, cmake, gcc, Terraform, kubectl, Helm
- Package managers: npm, pip, gem, cargo, composer

Full software manifests are published at github.com/actions/runner-images — check these when you need to know exactly what's available without installing it yourself.

GitHub-Hosted Runner Specifications

Specification	Value
---------------	-------

vCPUs	2 cores (Linux/Windows), 3 cores (macOS)
RAM	7 GB (Linux/Windows), 14 GB (macOS)
Storage	14 GB SSD (Linux/Windows), 14 GB (macOS)
Network	Outbound internet access; no fixed IP address
Persistence	None — fresh VM per job, all data erased after job completes
Lifetime	6 hours maximum (or 72 hours for self-hosted)
Cost (public repos)	Free — unlimited minutes
Cost (private repos)	Linux: 1x minute rate Windows: 2x macOS: 10x

Important Limitation: No Fixed IP Address

GitHub-hosted runners have dynamic, rotating IP addresses.

This means you cannot whitelist runner IPs in firewalls or database security groups.

If your workflow needs to reach a database, API, or server behind an IP whitelist, you have two options: (1) use a self-hosted runner on a known IP, or (2) use GitHub's larger runners feature which supports static IP ranges (available on Team/Enterprise plans).

Self-Hosted Runners

A self-hosted runner is a machine you own and control — a server, VM, cloud instance, Raspberry Pi, or even your laptop — that you register with GitHub so your workflows can run on it. You install the GitHub Actions runner application, which polls GitHub for jobs and executes them locally.

Why Use Self-Hosted Runners?

- Network access: Your runner is inside your network, so it can reach databases, internal APIs, and services behind firewalls — without exposing them to the internet
- Fixed IP address: If your workflows need to talk to whitelisted resources (AWS RDS, internal Kubernetes clusters, protected APIs), a self-hosted runner has a known, stable IP
- Custom hardware: Need GPU for ML model training? A high-memory server for large test suites? Special hardware like Apple Silicon for iOS builds? Self-hosted runners give you this
- Larger storage: GitHub-hosted runners have ~14 GB. Self-hosted runners have whatever disk you provision
- Cost at scale: For very high-volume private repos, self-hosted runners can be significantly cheaper than paying per-minute for GitHub-hosted runners
- Persistent tool cache: Since the machine isn't destroyed after each job, your npm/pip/Docker caches persist across runs — faster cold starts

Tradeoffs and Responsibilities

- You manage OS updates, security patches, and the runner software itself
- You are responsible for security — a compromised self-hosted runner could expose your entire network
- You must ensure the machine has the software tools your workflows need
- Scaling requires either multiple machines or a runner management layer (like Actions Runner Controller)

Setting Up a Self-Hosted Runner

GitHub provides a step-by-step script in your repository or organization settings. Here is what the installation looks like on a Linux server:

```
# On your Linux server / VM

# Step 1: Create a dedicated user for the runner (security best practice)
sudo useradd -m github-runner
sudo su - github-runner

# Step 2: Create a directory for the runner
mkdir actions-runner && cd actions-runner

# Step 3: Download the latest runner package
curl -o actions-runner-linux-x64-2.321.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.321.0/\
  actions-runner-linux-x64-2.321.0.tar.gz

# Step 4: Extract
tar xzf ./actions-runner-linux-x64-2.321.0.tar.gz

# Step 5: Configure (token from: Settings → Actions → Runners → New runner)
./config.sh \
  --url https://github.com/YOUR-ORG/YOUR-REPO \
  --token YOUR_REGISTRATION_TOKEN \
  --name my-production-runner \
  --labels self-hosted,linux,production,high-memory \
  --work _work

# Step 6: Install and start as a system service (runs on boot)
sudo ./svc.sh install
sudo ./svc.sh start

# Verify it is running
sudo ./svc.sh status
```

Using a Self-Hosted Runner in a Workflow

Reference your self-hosted runner using the labels you assigned during setup. You can target by general labels or very specific ones.

```
jobs:
  # Option 1: Target any self-hosted runner
  build-basic:
    runs-on: self-hosted

  # Option 2: Target a self-hosted Linux runner
  build-linux:
    runs-on: [self-hosted, linux]

  # Option 3: Target a specific labeled runner
  deploy-production:
    runs-on: [self-hosted, linux, production]
    # Only runs on the runner registered with the 'production' label
    # Ensures production deployments only run on your approved server

  # Option 4: Mix - GitHub-hosted for CI, self-hosted for deployment
  test:
    runs-on: ubuntu-latest          # Fast, clean GitHub-hosted runner
    steps:
      - run: npm test

  deploy:
    needs: test
    runs-on: [self-hosted, linux, production] # Self-hosted with network access
    steps:
      - run: ./deploy-to-internal-cluster.sh
```

Runner Groups: Organisation-Level Control

On GitHub Teams and Enterprise plans, you can create runner groups — pools of runners with controlled access. For example, you can create a 'Production Runners' group that is only accessible to specific repositories. This prevents a developer repository from accidentally (or maliciously) using a runner that has production credentials.

```
# Reference a runner group in a workflow
jobs:
  deploy:
    runs-on:
      group: production-runners # Runner group name
      labels: [linux, high-memory] # Optional: filter within the group
```

Scaling Self-Hosted Runners: Actions Runner Controller (ARC)

For teams running many workflows, manually managing a fixed pool of self-hosted runners is inefficient. Actions Runner Controller (ARC) is a Kubernetes operator that automatically scales runners up and down based on demand — similar to how cloud autoscaling works for application servers.

- Runs on your Kubernetes cluster (EKS, GKE, AKS, or on-premises)
- Spins up a new runner pod when a job is queued; destroys it when the job completes
- Each job gets a completely fresh container — same isolation guarantee as GitHub-hosted runners
- Dramatically reduces cost compared to running idle servers
- Ideal for organisations with spiky, unpredictable CI/CD workloads

Side-by-Side: GitHub-Hosted vs Self-Hosted Runners

Factor	GitHub-Hosted Runners	Self-Hosted Runners
Managed by	GitHub (zero ops for you)	You (full responsibility)
Setup time	Zero — just write runs-on:	30-60 minutes initial setup
Cost model	Per-minute (free for public repos)	Server cost; no per-minute charge
IP address	Dynamic / rotating	Fixed / known
Network access	Public internet only	Internal network + internet
Hardware	2 vCPU, 7 GB RAM (standard)	Whatever you provision
GPU support	No (standard runners)	Yes — attach any GPU
Software	Large pre-installed toolkit	You install what you need
Persistence	Fresh VM per job	Configurable — can persist or not
Scaling	Automatic (infinite capacity)	Manual or via ARC on Kubernetes
Security	Managed by GitHub	You secure the machine and network
Best for	Most CI/CD workloads	Private networks, GPU, scale cost

4.4 Workflow YAML Structure

Every GitHub Actions workflow file follows this structure:

```
name: Workflow Name # Display name in GitHub UI

on: # Trigger(s)
  push:
    branches: [main]
  pull_request:
    branches: [main]

env: # Global environment variables
```



```
NODE_VERSION: '18'

jobs:
  job-name:                # Job ID (used for dependencies)
    name: Human-readable Job Name # Display name
    runs-on: ubuntu-latest    # Runner OS

    steps:                  # Sequence of steps
      - name: Step Name
        uses: actions/checkout@v4 # Use an Action

      - name: Another Step
        run: echo 'Hello World'    # Run a shell command
```

Chapter 5: BEGINNER — Your First Workflows

Example 1: Hello World Workflow

Let's start with the absolute minimum. This workflow runs every time code is pushed to the main branch and prints a message.

```
# .github/workflows/hello-world.yml

name: Hello World

on:
  push:
    branches: [main]

jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - name: Print greeting
        run: echo 'Hello from GitHub Actions!'

      - name: Print date
        run: date

      - name: Print environment info
        run: |
          echo "OS: $RUNNER_OS"
          echo "Repo: $GITHUB_REPOSITORY"
          echo "Branch: $GITHUB_REF_NAME"
          echo "Commit: $GITHUB_SHA"
```

Key Concepts Here:

run: | (pipe) lets you write multi-line shell scripts

GitHub provides default environment variables like `GITHUB_REPOSITORY`, `GITHUB_SHA`, `RUNNER_OS`

Each step gets a name — this shows up in the GitHub Actions UI for easy debugging

Example 2: Basic Node.js CI Pipeline

This is the most common starting point for JavaScript/TypeScript projects. It installs dependencies, runs the linter, and runs tests on every push and pull request.

```
# .github/workflows/nodejs-ci.yml

name: Node.js CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Install, Lint, and Test
    runs-on: ubuntu-latest

    steps:
      # Step 1: Check out the repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # Step 2: Set up Node.js
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: 'npm' # Cache npm dependencies automatically

      # Step 3: Install dependencies
      - name: Install dependencies
        run: npm ci # Use 'ci' instead of 'install' for reproducibility

      # Step 4: Run linter
      - name: Run ESLint
        run: npm run lint

      # Step 5: Run tests
      - name: Run tests
        run: npm test
```

Why npm ci instead of npm install?

npm ci installs exactly what's in package-lock.json — no updates, no surprises

It's faster in CI environments and guarantees reproducible builds

Always use npm ci in pipelines; use npm install only locally when updating deps

Example 3: Python CI Pipeline

For Python projects — install dependencies, run type checking, and run pytest.

```
# .github/workflows/python-ci.yml

name: Python CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest mpy flake8

      - name: Lint with flake8
        run: flake8 src/ --max-line-length=100

      - name: Type check with mpy
        run: mpy src/

      - name: Run tests
        run: pytest tests/ -v --tb=short
```

Example 4: Using Secrets and Environment Variables

Secrets are encrypted values stored in GitHub (Settings → Secrets → Actions). Never hardcode API keys, passwords, or tokens in your workflow files.

```
# .github/workflows/with-secrets.yml

name: Deploy with Secrets

on:
  push:
    branches: [main]

env:
  APP_NAME: my-application          # Non-sensitive global variable

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Deploy to server
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL } # From GitHub Secrets
          API_KEY: ${ secrets.API_KEY }
          ENVIRONMENT: production
        run: |
          echo "Deploying $APP_NAME to $ENVIRONMENT"
          # GitHub automatically masks secrets in logs
          ./scripts/deploy.sh

      - name: Send notification
        if: success()
        env:
          SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
        run: |
          curl -X POST $SLACK_WEBHOOK \
            -H 'Content-Type: application/json' \
            -d '{"text": "Deployment successful!  "}'
```

Security Best Practices for Secrets:

Add secrets at: Repository Settings → Secrets and variables → Actions → New repository secret

Secrets are masked in logs — GitHub replaces them with *** automatically

Use environment secrets for production-only values (extra protection layer)

Never use secrets in environment names or job names — those are not masked

Rotate secrets regularly and audit who has admin access to the repository

Chapter 6: INTERMEDIATE — Multi-Job Pipelines

Example 5: Sequential Jobs with Dependencies

Real pipelines have multiple jobs that run in a specific order. The `needs:` keyword defines dependencies between jobs. If a dependency fails, downstream jobs are automatically skipped.

```
# .github/workflows/full-pipeline.yml

name: Full CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Job 1: Run tests
  test:
    name: Test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm' }
      - run: npm ci
      - run: npm test

  # Job 2: Code quality (runs PARALLEL to test)
  quality:
    name: Code Quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check

  # Job 3: Build Docker image (waits for test AND quality)
  build:
    name: Build Docker Image
    runs-on: ubuntu-latest
    needs: [test, quality] # Both must pass first
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .

  # Job 4: Deploy (only on main branch, after build)
  deploy:
```

```
name: Deploy to Production
runs-on: ubuntu-latest
needs: build
if: github.ref == 'refs/heads/main'    # Only on main branch
steps:
  - uses: actions/checkout@v4
  - name: Deploy
    run: echo 'Deploying to production...'
```

Example 6: Matrix Builds — Test Multiple Versions

Matrix builds let you run the same job across multiple configurations simultaneously. This is perfect for testing against multiple Node.js versions, operating systems, or database versions.

```
# .github/workflows/matrix-build.yml

name: Matrix Build

on: [push, pull_request]

jobs:
  test:
    name: Test on Node ${ matrix.node } / ${ matrix.os }
    runs-on: ${ matrix.os }

    strategy:
      matrix:
        node: [16, 18, 20]
        os: [ubuntu-latest, windows-latest, macos-latest]
        exclude:
          # Skip Windows + Node 16 combination
          - os: windows-latest
            node: 16
      fail-fast: false    # Don't cancel all jobs if one fails

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js ${ matrix.node }
        uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node }
          cache: 'npm'

      - run: npm ci
      - run: npm test
```

Example 7: Artifacts — Sharing Data Between Jobs

Jobs run on separate, isolated runners. To share files between jobs (e.g., pass a build artifact from the build job to the deploy job), you must upload and download artifacts.

```
# .github/workflows/artifacts.yml

name: Build and Deploy with Artifacts

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm' }
      - run: npm ci && npm run build

      # Upload the build output as an artifact
      - name: Upload build artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist/
          retention-days: 7      # Keep for 7 days

  deploy:
    runs-on: ubuntu-latest
    needs: build
    steps:
      # Download the artifact from the build job
      - name: Download build artifacts
        uses: actions/download-artifact@v4
        with:
          name: dist-files
          path: dist/

      - name: Deploy
        run: |
          ls -la dist/
          echo 'Deploying dist/ to production server...'
```

Example 8: Caching Dependencies

Caching dramatically speeds up pipelines by saving and restoring dependencies across workflow runs. Without caching, npm install runs every time from scratch.

```
# .github/workflows/with-cache.yml

name: CI with Caching
```

```
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Method 1: Let setup-node handle caching (recommended for npm/yarn/pnpm)
      - uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: 'npm' # Automatically caches based on package-lock.json

      # Method 2: Manual cache control (for custom paths)
      - name: Cache custom build outputs
        uses: actions/cache@v4
        with:
          path: |
            ~/.npm
            ${{ github.workspace }}/.next/cache
          key: ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-
            ${{ hashFiles('**.[jt]s', '**.[jt]sx') }}
          restore-keys: |
            ${{ runner.os }}-nextjs-${{ hashFiles('**/package-lock.json') }}-
            ${{ runner.os }}-nextjs-

      - run: npm ci
      - run: npm run build
```

⚡ Cache Key Strategy:

The cache key determines when the cache is invalidated

`hashFiles('**/package-lock.json')` creates a hash of your lock file

If `package-lock.json` changes (new deps added), cache key changes, cache is rebuilt

`restore-keys` provides fallback patterns if exact key isn't found

Good caching can reduce a 5-minute npm install to under 30 seconds

Example 9: Conditional Steps and Jobs

Use `if:` conditions to control which steps or jobs run based on branch, event type, previous step results, or custom expressions.

```
# .github/workflows/conditionals.yml

name: Conditional Workflow

on: [push, pull_request]
```



```
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

      # Only runs if tests PASSED
      - name: Generate coverage report
        if: success()
        run: npm run coverage

      # Always runs, even if tests FAILED
      - name: Upload logs
        if: always()
        run: ./upload-logs.sh

      # Only runs if tests FAILED
      - name: Notify on failure
        if: failure()
        run: ./notify-team.sh

# Only deploys from the main branch
deploy-staging:
  needs: ci
  runs-on: ubuntu-latest
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  steps:
    - name: Deploy to staging
      run: echo 'Deploying to staging'

# Only deploys on version tags (e.g., v1.2.3)
deploy-production:
  needs: ci
  runs-on: ubuntu-latest
  if: startsWith(github.ref, 'refs/tags/v')
  steps:
    - name: Deploy to production
      run: echo 'Deploying to production'
```

Chapter 7: ADVANCED — Production-Grade Patterns

Example 10: Docker Build, Tag & Push to Registry

Building and pushing Docker images is one of the most common CI/CD tasks. This example builds an image, tags it with the Git SHA and 'latest', and pushes to GitHub Container Registry (GHCR).

```
# .github/workflows/docker-build-push.yml

name: Build and Push Docker Image

on:
  push:
    branches: [main]
    tags: ['v*.*.*']

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write      # Required to push to GHCR

    steps:
      - uses: actions/checkout@v4

      # Set up Docker Buildx (for multi-platform builds)
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      # Log in to GitHub Container Registry
      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN } # Auto-generated token!

      # Generate image tags and labels
      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
          tags: |
            type=ref,event=branch
            type=semver,pattern={{version}}
            type=sha,prefix=sha-
```

```
# Build and push (uses BuildKit layer cache)
- name: Build and push Docker image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Example 11: Deploy to AWS ECS with OIDC (No Secrets!)

This is the modern, secure approach to deploying to AWS. Instead of storing long-lived AWS access keys as secrets (a security risk), we use OpenID Connect (OIDC) to get short-lived credentials. No keys to rotate, no keys to leak.

```
# .github/workflows/deploy-ecs.yml
# PREREQUISITE: Configure AWS IAM OIDC provider for GitHub Actions

name: Deploy to AWS ECS

on:
  push:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: imisi-health-api
  ECS_SERVICE: imisi-api-service
  ECS_CLUSTER: imisi-production
  CONTAINER_NAME: api

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write # Required for OIDC
      contents: read

    steps:
      - uses: actions/checkout@v4

      # Get short-lived AWS credentials via OIDC (no stored keys!)
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/GitHubActionsRole
          aws-region: ${{ env.AWS_REGION }}

      # Log in to ECR
      - name: Login to Amazon ECR
        id: login-ecr
```

```
uses: aws-actions/amazon-ecr-login@v2

# Build, tag, and push to ECR
- name: Build, tag, and push image to ECR
  id: build-image
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.sha }
  run: |
    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
    echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT

# Update ECS task definition with new image
- name: Download ECS task definition
  run: aws ecs describe-task-definition --task-definition imisi-api --query
taskDefinition > task-def.json

- name: Update ECS task definition with new image
  id: task-def
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: task-def.json
    container-name: ${ env.CONTAINER_NAME }
    image: ${ steps.build-image.outputs.image }

# Deploy to ECS
- name: Deploy to ECS
  uses: aws-actions/amazon-ecs-deploy-task-definition@v2
  with:
    task-definition: ${ steps.task-def.outputs.task-definition }
    service: ${ env.ECS_SERVICE }
    cluster: ${ env.ECS_CLUSTER }
    wait-for-service-stability: true
```

Example 12: Reusable Workflows

Reusable workflows let you define a workflow once and call it from multiple other workflows. This is the GitHub Actions equivalent of functions — DRY (Don't Repeat Yourself) for CI/CD.

```
# .github/workflows/reusable-deploy.yml (the reusable workflow)

name: Reusable Deploy

on:
  workflow_call: # This makes it callable from other workflows
  inputs:
    environment:
      required: true
      type: string
    image-tag:
      required: true
      type: string
```

```
secrets:
  deploy-key:
    required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${ inputs.environment }
    steps:
      - name: Deploy ${ inputs.image-tag } to ${ inputs.environment }
        env:
          DEPLOY_KEY: ${ secrets.deploy-key }
        run: |
          echo "Deploying ${ inputs.image-tag } to ${ inputs.environment }"

---

# .github/workflows/main-pipeline.yml (the caller)

name: Main Pipeline
on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${ steps.tag.outputs.tag }
    steps:
      - uses: actions/checkout@v4
      - id: tag
        run: echo "tag=${ github.sha }" >> $GITHUB_OUTPUT

  deploy-staging:
    needs: build
    uses: ../github/workflows/reusable-deploy.yml
    with:
      environment: staging
      image-tag: ${ needs.build.outputs.image-tag }
    secrets:
      deploy-key: ${ secrets.STAGING_KEY }

  deploy-production:
    needs: deploy-staging
    uses: ../github/workflows/reusable-deploy.yml
    with:
      environment: production
      image-tag: ${ needs.build.outputs.image-tag }
    secrets:
      deploy-key: ${ secrets.PRODUCTION_KEY }
```

Example 13: Environment Protection Rules

GitHub Environments allow you to require manual approval before deploying to sensitive environments like production. This adds a human checkpoint in your automated pipeline.

```
# Setup: Go to Repo Settings → Environments → Create 'production'
# Enable: Required reviewers (add specific people or teams)
# Enable: Wait timer (optional delay in minutes)
# Enable: Deployment branch policy (only deploy from 'main')

# .github/workflows/deploy-with-approval.yml

name: Deploy with Approval Gate

on:
  push:
    branches: [main]

jobs:
  deploy-staging:
    name: Deploy to Staging
    runs-on: ubuntu-latest
    environment: staging # No protection on staging
    steps:
      - run: echo 'Deploying to staging automatically'

  deploy-production:
    name: Deploy to Production
    runs-on: ubuntu-latest
    needs: deploy-staging
    environment: production # REQUIRES APPROVAL from reviewers
    steps:
      - run: echo 'Deploying to production after approval'
```

Example 14: Full NestJS + PostgreSQL CI Pipeline

This is a complete, production-realistic pipeline for a NestJS API with a PostgreSQL database — very relevant for teams building Node.js backends like the Imisi Health API.

```
# .github/workflows/nestjs-ci.yml

name: NestJS CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  lint-and-type-check:
    name: Lint & Type Check
    runs-on: ubuntu-latest
```

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: '18', cache: 'npm' }
  - run: npm ci
  - run: npm run lint
  - run: npm run build # TypeScript type check via tsc

unit-tests:
  name: Unit Tests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '18', cache: 'npm' }
    - run: npm ci
    - run: npm run test
    - run: npm run test:cov

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v4
      with:
        token: ${ secrets.CODECOV_TOKEN }
```

```
integration-tests:
  name: Integration Tests (with PostgreSQL)
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:15
      env:
        POSTGRES_PASSWORD: testpassword
        POSTGRES_DB: imisi_test
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

    redis:
      image: redis:7
      ports:
        - 6379:6379

  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '18', cache: 'npm' }
    - run: npm ci

    - name: Run database migrations
      env:
        DATABASE_URL: postgresql://postgres:testpassword@localhost:5432/imisi_test
      run: npm run migration:run

    - name: Run integration tests
      env:
```

```
DATABASE_URL: postgresql://postgres:testpassword@localhost:5432/imisi_test
REDIS_URL: redis://localhost:6379
JWT_SECRET: test-secret-key
run: npm run test:e2e
```

Example 15: Automated Security Scanning

Security scanning should run on every pull request. This example uses CodeQL (GitHub's security analysis engine) and Snyk for dependency vulnerability scanning.

```
# .github/workflows/security.yml

name: Security Scanning

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 6 * * 1' # Every Monday at 6 AM UTC

jobs:
  codeql:
    name: CodeQL Analysis
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      actions: read
      contents: read

    strategy:
      matrix:
        language: [javascript-typescript]

    steps:
      - uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
        with:
          languages: ${{ matrix.language }}

      - uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm' }
      - run: npm ci && npm run build

      - name: Perform CodeQL Analysis
        uses: github/codeql-action/analyze@v3

  dependency-scan:
    name: Dependency Vulnerability Scan
    runs-on: ubuntu-latest
```



```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: '18', cache: 'npm' }
  - run: npm ci

  - name: Run npm audit
    run: npm audit --audit-level=high

  - name: Run Snyk security scan
    uses: snyk/actions/node@master
    env:
      SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
    with:
      args: --severity-threshold=high

secret-scan:
  name: Secret Scanning
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0 # Full history for secret scanning

    - name: TruffleHog Secret Scan
      uses: trufflesecurity/trufflehog@main
      with:
        path: ./
        base: ${ github.event.repository.default_branch }
```

Example 16: Custom Composite Action

When you find yourself repeating the same group of steps across multiple workflows, create a custom composite action. This is stored in your repository and referenced like any marketplace action.

```
# .github/actions/setup-and-install/action.yml
# This is a custom COMPOSITE ACTION in your repo

name: 'Setup Node and Install Dependencies'
description: 'Sets up Node.js with caching and installs npm dependencies'

inputs:
  node-version:
    description: 'Node.js version to use'
    required: false
    default: '18'

runs:
  using: 'composite'
  steps:
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
```

```
    node-version: ${ inputs.node-version }
    cache: 'npm'

  - name: Install dependencies
    shell: bash
    run: npm ci

  - name: Print setup summary
    shell: bash
    run: |
      echo "Node version: $(node --version)"
      echo "npm version: $(npm --version)"

---

# Usage in any workflow:
steps:
  - uses: actions/checkout@v4
  - uses: ./github/actions/setup-and-install # Local action
    with:
      node-version: '20'
  - run: npm test
```

Example 17: Slack Notifications with Rich Messages

Keeping your team informed about deployments and failures is critical. This example sends rich, contextual Slack notifications.

```
# .github/workflows/notify.yml

name: CI with Notifications

on:
  push:
    branches: [main]

jobs:
  build-and-notify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm' }
      - run: npm ci && npm test

      - name: Notify Slack on Success
        if: success()
        uses: slackapi/slack-github-action@v2
        with:
          webhook: ${ secrets.SLACK_WEBHOOK_URL }
          webhook-type: incoming-webhook
          payload: |
            {
```

```
      "text": "✅ *Build Passed* - ${ github.repository }",
      "attachments": [{
        "color": "good",
        "fields": [
          {"title": "Branch", "value": "${ github.ref_name }", "short": true},
          {"title": "Commit", "value": "${ github.sha }", "short": true},
          {"title": "Author", "value": "${ github.actor }", "short": true}
        ]
      }]
    }

- name: Notify Slack on Failure
  if: failure()
  uses: slackapi/slack-github-action@v2
  with:
    webhook: ${ secrets.SLACK_WEBHOOK_URL }
    webhook-type: incoming-webhook
    payload: |
      {
        "text": " ❌ *Build Failed* - ${ github.repository }",
        "attachments": [{
          "color": "danger",
          "fields": [
            {"title": "Branch", "value": "${ github.ref_name }", "short": true},
            {"title": "Run URL", "value":
"${ github.server_url }/${ github.repository }/actions/runs/${ github.run_id }"}
          ]
        }]
      }
    }
```

PART 4: BEST PRACTICES & NEXT STEPS

Chapter 8: GitHub Actions Best Practices

8.1 Security Best Practices

- Always pin actions to a specific commit SHA, not just a tag (e.g., uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683)
- Use GITHUB_TOKEN for GitHub operations instead of personal access tokens where possible
- Use OIDC for cloud provider authentication — eliminate long-lived cloud credentials
- Set minimum required permissions using the permissions: block at the job level
- Never print secrets in run: steps — use environment variables
- Enable secret scanning and push protection in your repository settings

8.2 Performance Best Practices

- Cache aggressively — npm, pip, gradle, maven all support caching via actions/cache
- Run jobs in parallel when they don't depend on each other
- Use fail-fast: false in matrix builds when you need all results, not just the first failure
- Split slow test suites using matrix sharding (split tests across N parallel jobs)
- Use docker layer caching (cache-from: type=gha) for Docker builds

8.3 Maintainability Best Practices

- Use environment variables at the top of workflows for all configurable values
- Create reusable workflows for patterns repeated across multiple repos
- Write descriptive names for all jobs and steps — they're your pipeline documentation
- Use workflow_dispatch with inputs for manual deployment workflows with parameters
- Keep workflows focused — one workflow per concern (CI, CD, security, releases)

8.4 Common Mistakes to Avoid

✗ Anti-Patterns to Avoid

Storing AWS access keys as secrets — use OIDC instead

Running everything in one massive job — parallelize!

Not caching dependencies — wastes minutes on every run

Using latest tags for actions (actions/checkout@latest) — use version tags or SHAs

Skipping security scanning because 'it slows the pipeline' — security debt is real

Deploying to production without any approval gate or environment protection

Not testing the pipeline itself — add a workflow_dispatch trigger to test manually

Chapter 9: Exercises for Students

Work through these exercises in order. Each builds on the previous.

Beginner Exercises

1. Create a repository on GitHub with a simple Node.js or Python app. Write a workflow that runs on every push and prints the Git commit SHA, branch name, and current date.
2. Add a test step: install Jest (Node.js) or pytest (Python), write at least 3 tests, and have the workflow fail if any test fails.
3. Add a secret called APP_MESSAGE to your repository. Update the workflow to read this secret and echo it in a step.

Intermediate Exercises

4. Create a 3-job pipeline: lint → test → build. Use needs: to enforce order. The build job should upload its output as an artifact.
5. Set up a matrix build that runs your tests on Node.js versions 16, 18, and 20. Use fail-fast: false.
6. Add a conditional deploy job that only runs when pushing to the main branch, and sends a Slack (or Discord) notification on success or failure.

Advanced Exercises

7. Write a Dockerfile for your application. Create a workflow that builds the image, tags it with the commit SHA, and pushes it to GitHub Container Registry.
8. Create a reusable workflow that accepts an 'environment' input parameter and simulates deploying to staging vs. production.

9. Add CodeQL security scanning to your repository. Create a separate scheduled workflow that runs on Sundays at midnight to audit dependencies.
10. Implement a full GitOps workflow: push to a feature branch triggers CI; merging to main triggers staging deployment; creating a version tag (v1.0.0) triggers production deployment with an approval gate.

Chapter 10: Learning Resources & Next Steps

Official Documentation

- GitHub Actions Documentation: docs.github.com/en/actions
- GitHub Actions Marketplace: github.com/marketplace?type=actions
- GitHub Skills (free interactive learning): skills.github.com

Recommended Learning Path After This Course

11. Infrastructure as Code: Learn Terraform to provision your CI/CD infrastructure itself as code
12. GitOps: Learn ArgoCD or Flux for Kubernetes-based continuous deployment
13. Monitoring & Observability: Integrate pipeline metrics into Datadog or Grafana
14. Advanced Security: Study OWASP Top 10, implement DAST in your pipeline
15. AWS DevOps Professional: Pursue the AWS DevOps Engineer Professional certification

Final Thought: The Culture Behind CI/CD

CI/CD is not just tooling — it's a culture shift. The tools are the easy part.

The hard part is committing to small, frequent changes; writing tests before features; treating a broken build as an emergency; and empowering every developer to deploy. Master these habits, and the tools become second nature.

Keep building. Keep shipping. — Borderless Tech Academy